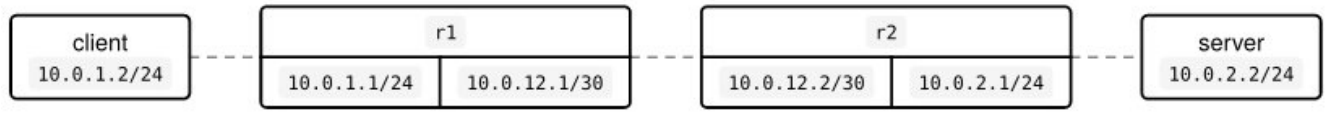
PR Rechnernetze und Netzwerkprogrammierung 621.801/621.802

Institut für Informationstechnologie (ITEC) · WS 2026/2027

Übungsblatt 06

IP, Subnetting und Routing mit Mininet

Ausgangstopologie (start_lab.py)



Ü 6.1 IP-Adressen und Interfaces

Analysieren Sie die Interfaces und IP-Adressen von client, r1, r2 und server mit den folgenden Befehlen:

```
ip -br addr
ip addr
```

- Erstellen Sie eine Tabelle mit Hostname, Interface, IP-Adresse, Prefix und zugehörigem Netzwerk.
- Erklären Sie, welche Interfaces zum selben IP-Netz gehören und welche durch Router getrennt sind.

Ü 6.2 Routingtabellen verstehen

Untersuchen Sie die Routingtabellen. Erklären Sie wie Daten von 10.0.1.2 nach 10.0.2.2 geleitet werden.

```
ip route
```

Erstellen Sie eine kleine ***Forwarding-Tabelle*** für r1 und r2 und geben Sie jeweils den Next Hop an.

Ü 6.3 Statische Routen

Entfernen Sie auf r1 und r2 die bereitgestellten Routen und konfigurieren Sie diese anschließend manuell neu. Dokumentieren Sie jeden Schritt.

```
ip route
ip route del <route>
ip route add <destination> via <next-hop>
```

Testen Sie die Verbindung anschließend mit einem Ping von Client zu Server:

```
ping -c 3 10.0.2.2
```

Ü 6.4 IPv4-Subnetting und Adressplanung

Planen Sie für ein fiktives Netzwerk Subnet für 200 Endgeräte, 50 Server, 20 Management-Geräte (Router, Access-Points,...) und eine P2P (Punkt-zu-Punkt) Verbindung zwischen zwei Routern.

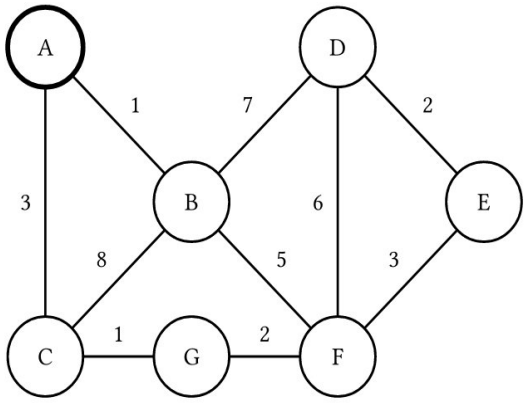
Der Plan sollte mindestens folgende Attribute enthalten:

- Netzadresse
- Prefix/Subnetzmaske
- erste nutzbare Hostadresse
- letzte nutzbare Hostadresse
- Broadcast-Adresse
- Anzahl möglicher Hosts

Vergleichen Sie Ihren Plan mit der RN-Lab-Topologie (start_lab.py) und begründen Sie die gewählten Prefixlängen.

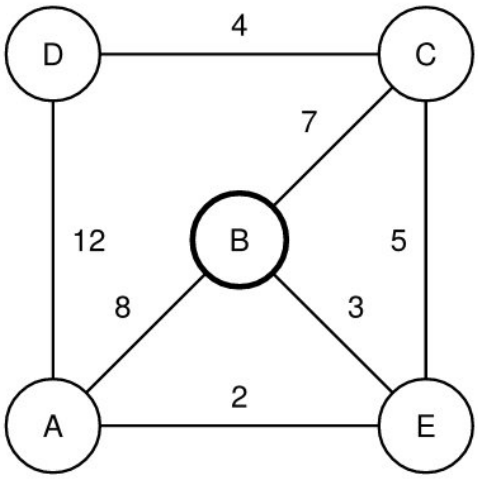
Ü 6.5 Vermittlungsschicht und Routing: *Link-State-Algorithmus*

Gegeben sei ein Netzwerk wie in **Abbildung 1** dargestellt. Berechnen Sie, mithilfe des Algorithmus von Dijkstra, den kürzesten Pfad ausgehend von **Knoten `A`** zu **allen Netzwerkknoten**. Benutzen Sie dazu eine ähnliche Tabelle, wie Sie sie in der Vorlesung kennengelernt haben. Die Zahlen geben die Kosten für die jeweilige Kante bzw. den Link an.



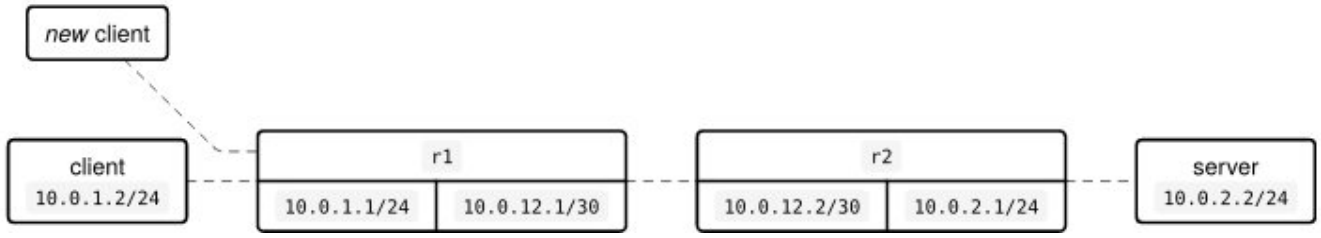
Ü 6.6 Vermittlungsschicht und Routing: *Distance-Vector-Algorithmus*

Gegeben sei ein Netzwerk wie in Abbildung 2 dargestellt. In diesem Netzwerk kennt jeder Knoten die Kosten der Verbindungen zu seinen unmittelbaren Nachbarn. Wenden Sie den Distance-Vector-Algorithmus an und geben Sie sowohl Distanz-Tabellen von Knoten B nach jedem Schritt als auch die initiale Tabelle an. Beachten Sie dabei, dass der Distance-Vector mit der Kante, welche die geringsten Kosten hat, beginnt. Markieren Sie in den Distanztabellen die jeweiligen Änderungen und geben Sie auch den erhaltenen Distanzvektor in jedem Schritt an.



Ü 6.7 Mininet-Topologie erweitern

Bearbeiten Sie ab jetzt topology.py. Fügen Sie nun einen weiteren Client dem Netzwerk hinzu. Die Python-Anwendung (*tcp_client.py, tcp_server.py*) darf dabei nicht verändert werden.



- Host anlegen
- geeignete IP-Adresse vergeben
- mit r1 verbinden
- Default Route setzen
- Verbindung zum Server testen

Dokumentieren Sie die Änderungen, die Sie vorgenommen haben, und erklären Sie, warum tcp_client.py und tcp_server.py unverändert bleiben können.

Anhang: Dateien

Datei	Funktion	Bearbeitung
topology.py	vorgegebene Mehr-Router-Topologie	nicht verändern
start_lab.py	Startet Umgebung und Konsolen	nicht verändern
inspect_network.sh	Netzwerkzustand anzeigen	verwenden
capture_interface.sh	PCAP-Mitschnitt	verwenden